I AKSHAYA SEKAR (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: AKSHAYA SEKAR

SIMATS ENGINEERING

CO2_AT3 ERROR ANALYSIS TASK

COURSE NAME: Software Testing

COURSE CODE: CSA3711

COURSE OUTCOME COVERED: CO2

Assessment Tool Weightage: 20%

1. Question: A tester designs test cases for an online shopping system using equivalence partitioning but incorrectly groups input ranges (e.g., valid and invalid inputs mixed in one class). Additionally, boundary values are not considered. Analyze the given test design and identify all errors in equivalence class partitioning and boundary value analysis. Examine the root causes of these mistakes with proper reasoning. Propose corrected test cases with proper classification and boundary coverage. Justify how your corrections improve defect detection effectiveness.

Answer: Equivalence Class Partitioning (ECP) requires inputs to be divided into distinct valid and invalid classes, with each class representing inputs expected to behave similarly. Mixing valid and invalid inputs in one class is incorrect because the test result cannot clearly establish whether validation works properly. Boundary Value Analysis (BVA) is also missing, so defects near minimum and maximum limits may remain undetected.

The root causes are poor understanding of input rules, incomplete analysis of valid ranges, and failure to identify boundary conditions. First, define each field's permitted range and format from the requirements. For a hypothetical quantity field that accepts integers from 1 to 100, use valid classes (1–100), invalid values below the minimum (less than 1), invalid values above the maximum (greater than 100), and invalid non-integer or empty inputs where applicable. Representative tests can include 0, 1, 2, 99, 100, and 101. These tests cover just below, at, and just above the limits. Separating classes and testing boundaries improves defect detection by exposing incorrect range checks and validation logic.

2. Question: A QA engineer performs random testing on a banking application and claims high coverage without using any structured techniques. Critical defects in transaction limits and boundary conditions are missed. Evaluate the testing approach and identify errors in strategy and coverage assumptions. Analyze the root causes behind missing critical defects. Suggest appropriate black box testing techniques to correct the approach. Justify how these techniques ensure better test of adequacy and reliability.

Answer: Random testing alone does not demonstrate adequate coverage. The approach lacks a documented relationship between requirements, risks, input classes, and test cases. It may repeatedly exercise ordinary inputs while missing transaction limits, invalid values, and important combinations. The root causes include an unstructured test selection process, insufficient risk analysis, and the assumption that many random tests guarantee coverage.

Use Equivalence Class Partitioning to select representative valid and invalid transaction amounts, and Boundary Value Analysis to test values below, at, and above transaction limits. Decision Table Testing can cover combinations such as account status, available balance, transaction type, and daily limit. State Transition Testing can verify states such as initiated, approved, declined, and reversed. Map tests to requirements, record expected results and prioritize high-risk financial functions. These black-box techniques provide systematic coverage and improve the likelihood of detecting limits, validation, and transaction-handling defects.

3. Question: A developer performs white box testing but creates an incomplete control flow graph, missing decision nodes and loops, resulting in poor path coverage. Analyze the errors in the control flow graph and path testing approach. Identify the root causes for inadequate test coverage. Propose corrections by constructing a proper control flow representation and identifying all independent paths. Justify how improved path testing ensures better coverage and defect detection.

Answer: A control flow graph (CFG) should represent each statement or basic block as a node and each possible transfer of control as an edge. Decision outcomes, including true and

false branches, and loop of entry, body, exit, and back edges must be included. Omitting decision nodes and loops makes the graph incomplete and causes the derived path tests to miss feasible execution routes.

The root causes are incomplete code analysis and confusion between statement coverage and path coverage. Reconstruct the CFG directly from the source code, mark every decision and loop, and identify entry and exit nodes. Then determine the independent paths using the graph's structure and, where appropriate, cyclomatic complexity. For a simple if–else structure, tests should exercise both the true and false outcomes. For a loop, test zero iterations when permitted, one iteration, multiple iterations, and termination at the limit. The exact independent paths depend on the actual program, which is not supplied in the question. A complete CFG and feasible path tests improve branch and loop coverage and help reveal control-flow defects.

4. Question: A testing team reports 100% statement coverage for a module but fails to detect logical errors in conditional statements. Evaluate the adequacy of the testing performed and identify gaps in coverage criteria. Analyze why statement coverage alone was insufficient. Recommend improved white box testing strategies such as branch or path coverage. Justify how the revised approach enhances overall test effectiveness and error detection.

Answer: One hundred percent statement coverage means every executable statement ran at least once; it does not prove that every decision outcome was tested. A test may execute the statements inside a conditional while never checking the opposite branch, allowing logical errors to remain hidden.

The gap is reliance on statement coverage as the only adequacy measure. Add branch coverage by testing each decision's true and false outcomes. Where practical, use condition coverage to make each Boolean condition evaluate both true and false, and use path coverage to exercise important feasible combinations of control flow. For example, for an if–else statement, create tests that cause the condition to be true and false and verify the expected

result in each case. For complex logic, select tests based on the code structure and risk. These criteria provide stronger evidence than statement coverage alone and improve detection of faulty conditions and omitted error-handling paths.

5. Question: A tester uses Equivalence Class Partitioning for an age-validation field (0–100) but defines only two classes: Negative numbers and Positive numbers. Analyze the errors in this partitioning strategy. Identify why this fails to account for valid vs. invalid age limits and the absence of Boundary Value Analysis. Propose a corrected set of partitions and justify how they ensure more robust input validation.

Answer: The partitioning is incomplete because positive numbers include both valid ages (1–100) and invalid ages above 100, while the value 0 is not clearly classified. It also does not address non-numeric, decimal, empty, or missing input if these are relevant to the requirements. Boundary Value Analysis is absent, so off-by-one errors may go unnoticed.

Use separate classes: valid ages from 0 through 100; invalid negative ages below 0; invalid ages above 100; and invalid formats or missing values where applicable. Boundary tests should include -1, 0, 1, 99, 100, and 101. Because the stated range explicitly includes 0–100, zero must be treated valid unless the requirement says otherwise. This classification separates expected behaviors and checks the limits directly, making input validation more robust.

6. Question: A developer claims full Condition Coverage for a complex `if (A & B) ` statement.

Answer: Condition coverage requires each individual Boolean condition to take both true and false values across the test set. For `if (A & B) `, a suitable set of tests is: A=true, B=true; A=false, B=true; and A=true, B=false. The first test makes both conditions true; the second makes A false; and the third makes B false. The combined decision is true only when both A and B are true.

The claim of full coverage should be checked against actual test inputs and results, because the assignment statement does not provide the developer's test cases. Decision coverage additionally requires the overall `if` decision to evaluate both true and false. The tests above

exercise both decision outcomes and both values for each condition. For safety- or risk-critical logic, stronger criteria such as modified condition/decision coverage may be appropriate where required. Documenting the conditions and expected outcomes makes the coverage claim verifiable.

7. Question: A QA team applies Decision Table Testing for a discount system but misses several combinations of "Premium Member" and "Coupon Applied," leading to price calculation errors. Analyze the errors in the construction of the decision table. Identify why the current approach is incomplete. Propose a revised, exhaustive decision table and justify its effectiveness in preventing production defects.

Answer: The decision table is incomplete because it omits one or more combinations of the two conditions: Premium Member (yes/no) and Coupon Applied (yes/no). Two binary conditions produce four possible combinations. The missing combinations can hide errors in discount precedence, eligibility, or price calculation.

A complete table should list all four rules: (1) premium=yes, coupon=yes; (2) premium=yes, coupon=no; (3) premium=no, coupon=yes; and (4) premium=no, coupon=no. For each rule, specify the expected discount and final price according to the actual business policy. The question does not state the discount percentages, whether discounts can be combined, or coupon eligibility rules, so those outcomes must be confirmed from requirements rather than invented. Test each rule with representative prices and verify the calculated total. The exhaustive table ensures that every stated condition combination is considered and helps prevent defects caused by missing rules or incorrect discount handling.

8. Question: During unit testing of a looping structure, a tester checks only the first and last iterations (n=1 and n=max). Defects occurring at n=0 and n=max+1 are missed. Evaluate the errors in this Loop Testing strategy. Analyze the root causes behind these omissions. Recommend a corrected approach for testing simple and nested loops and justify how it ensures path integrity.

Answer: The strategy is incomplete because it tests only the first and last permitted iterations

and omits important boundary cases. If zero iterations are valid, the loop should be tested with n=0. The behavior at n=max+1 should also be checked when it represents an invalid input or a limit violation. Testing only n=1 and n=max may miss incorrect initialization, termination, and limit checks.

For a simple loop, test zero iterations, one iteration, two iterations, a typical number of iterations, one less than the maximum, the maximum, and one more than the maximum where the specification defines that case. For nested loops, begin with the innermost loop while outer loops are held to minimal representative values, then expand testing to combinations and boundary conditions across the outer loops. Consider loop termination, counter updates, and exits. The precise tests depend on the loop's requirements. This systematic strategy improves confidence that loop entry, repetition, and termination behave correctly.

9. Question: An engineer utilizes Error Guessing as the primary testing technique for a safety-critical braking system, ignoring structured black box methods. Identify the errors in relying solely on ad-hoc techniques for high-risk modules. Analyze the impact on test adequacy. Suggest a combination of BVA and State Transition Testing to fix the strategy, justifying the increased safety assurance.

Answer: Relying only on Error Guessing is inadequate for a safety-critical braking system because the technique depends heavily on tester experience and may leave important requirements and conditions untested. It is difficult to demonstrate systematic coverage or repeatability from ad-hoc tests alone. The root problem is the absence of requirement-based, risk-driven test design, and explicit coverage criteria.

Use Boundary Value Analysis for specified thresholds such as speed, pressure, sensor ranges, or braking limits, testing values below, at, and above each defined boundary. Use State Transition Testing to check permitted and prohibited transitions among states defined by the system requirements, such as standby, braking requested, braking active, fault detected, and recovery, if those states are part of the actual design. Combine these structured methods with requirements of traceability, negative tests, independent review, and appropriate safety

assurance processes. The exact thresholds and states must come from the system specification. This combination improves systematic coverage and helps expose boundary and transition defects that error guessing alone may miss.

10. Question: A tester designs a Control Flow Graph (CFG) for a module but fails to represent the "else" branches of multiple nested statements, leading to 100% statement coverage but 0% branch coverage on error paths. Analyze the errors in graph construction. Identify the root causes for the missing nodes. Propose corrected CFG and independent paths to ensure branch adequacy.

Answer: The CFG is incomplete because it omits false outcomes from nested decisions and therefore fails to represent all possible control-flow routes, especially error paths. A statement-coverage result does not compensate for missing edges: statements may run while one or more decision outcomes remain untested. The root causes are incomplete review of the source code and failure to distinguish statement coverage from branch coverage.

Rebuild the CFG with a node for each basic block and decision, and edges for both true and false outcomes of every `if`, including nested conditions. Include error-handling blocks, merge points, entry and exit nodes, and any loops. Then derive independent feasible paths from the corrected graph and design tests to exercise each branch's outcome, including the error paths. Because the source code is not provided, the exact graph and independent paths cannot be enumerated accurately here. The corrected CFG should be checked against the implementation, and branch coverage should be measured after execution. This ensures the tests evaluate both normal and error control flows.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand